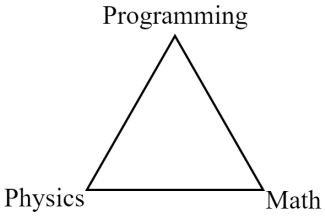
Lecture 2: Data representation

Lui Holder-Pearson

July 16, 2026

Learning Goals

- Understand the need to store numerical values in computers.
- Understand what sequences are, generically, and for data representation.
- Understand how groups of logical variables are used to encode numerical information — binary.
- Understand and interpret the representations of negative and hexadecimal numbers correlate to digital information.
- Understand the need to know which sequence is being used, and how this pertains to C programming.



Further Reading

Learning modules 1 & 2.
https://www.youtube.com/watch?v=1GSjbWt0c9M

References

Rizzoni and Kearns. *Principals and Applications of Electrical Engineering*, Chapters 13 & 14.

Version: 3.1

Numerical information as digital information

All information within computers is stored and handled as digital information.

Recall:

- Logical variables can be used to store logical values. These encompass a single `TRUE` or `FALSE` statement. Each value is stored within one individual *bit* of memory.

An example application is to determine whether or not to turn on the seatbelt warning light:

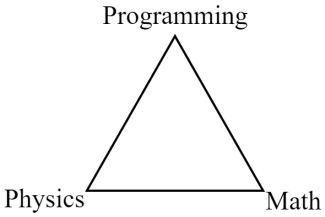


It is possible to have a single bit of memory to determine the seatbelt warning light:

- Digital information:
  "*The seatbelt warning light is on.*"
- Logic variable:
  "*seatbeltWarningLightStatus*"
- Logic value:
  "`True`"

For numerical information, however, we cannot have individual bits of memory for every single value that we want to represent, for example:

1 Sequences



Sequences of logical variables are simply any number of logical variables considered at the same time, to have a broader meaning: a group of variables which encode a greater meaning. Sequences are used extensively outside of computer engineering. For example, consider the lecture theatre: E8.

The possible number of combinations within a sequence is given:

$$M^N$$

Where M is the number of states each digit can have, and N is the number of digits in the sequence, or the length of the sequence.

In computer engineering, sequences of digital variables are often used extensively. Sequences can encode:

- Different operating states of a GPIO pin.
- The output from an ADC.
- Data, for example
  - Numbers
  - Characters

Table 12-1. Port Pin Configurations

DDxn	PORTxn	PUD (in MCUCR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

Figure 1: The sequence formed by `DDxn` `PORTxn` `PUD` and determines the status of the GPIO pin.

There are definitions for sequences of different lengths:

- **Bit**: One individual digit, either storing a `0` or a `1`.
- **Byte**: 8 bits. Specifically, eight consecutive bits that are addressed as a single sequence.
- **Word**: Less specific numerically, is the default number of bits that is manipulated by a microcontroller.

A nibble is half a byte.

We say that the information shown by a specific sequence is the *code*, or is *encoded* within the sequence.

1.1 Number representation

It is necessary to represent numbers within the CPU. This brings us into the math-iest chunk of CA: number bases.

We are accustomed to a base-10 system, where each digit of a sequence can take one of 10 digits. The higher the place in the sequence, the more the number is worth. For example, take the number:

0 1 2 3 4 5 6 7 8 9

$$23_{10}$$

If it was possible to store 10 different states within a single bit, it would therefore be possible to simply encode decimal numbers directly. Instead, seeing as there are only two possible states, `0` and `1`, we are forced to use a binary system.

As per decimal, we label the right-most logical variable the 0th digit, which is preceded by the 1st, and then the 2nd. Generally, the digit that is n digits to the left of the 0th digit is referred to as the nth digit:

$$d_4 d_3 d_2 d_1 d_0$$

The 0th bit is also referred to as the LSb, or least-significant bit, and the highest-value digital variable in the sequence as the MSb, or most-significant bit.

Instead of ones, tens, hundreds, etc., using base 2 gives us ones, twos, fours, eights etc, for example:

$$1110_2$$

Using binary encoding means that we can decode the value within the sequence in a systemic way:

$$Value = \sum d_n \times 2^n$$

1.2 Signed numbers

The manner we have used to encode numbers can be considered a sequence of n logical values, where each encapsulates the answer to the question:

Does the sum-total of my number include the value 2^n*?*

To represent negative numbers as well, one method would be to instead use the MSB to encode the answer to the question:

Is my number negative?, while leaving the rest of the bits to represent the absolute value of the number, called the *sign-magnitude* representation:

Pattern	Meaning
0 00	0
0 01	1
0 10	2
0 11	3
1 00	-0
1 01	-1
1 10	-2
1 11	-3

What is one shortfall of the sign-magnitude encoding of signed numbers?

Two's complement is an alternative, where the pattern that encodes -0 in the sign-magnitude representation, now holds the negative number of largest magnitude, equal to -2^{n-1}.

Pattern	Meaning
000	0
001	1
010	2
011	3
100	-4
101	-3
110	-2
111	-1

Two's complement is the most widely used representation of signed integers because it removes the duplicate zero, and lends itself to physical implementation, at the cost of human readability.

More formally:
$Val = (-1)^{d_n}\left(\sum_{n-1}(\overline{d_n} \times 2^n) + 1\right)$

2.6 The first tutorial will step you through some examples showing why two's complement is easily-implemented.

1.3 Conversion

It is important that we define the rules to convert between positive and negative numbers, i.e. how do we multiply by -1? For sign-magnitude this is fairly simple:

$$d_n = \overline{d_n}$$

For two's complement this is slightly more difficult:

1. Invert each digit.
2. Add one (e.g. 00000001_2 for an 8-bit number), ignoring overflow

For example: show the two's complement representation of -6 as a four-digit sequence:

The slightly less formulaic method is to find the right-most `1`, and then invert every digit to the **left of that**, excluding the `1` itself.

It is worth noting that several bits can be simultaneously manipulated with use of 'bitwise' operators: in C and Python these are:

`a` for NOT(a),
`a&b` for a AND b, and
`a|b` for a or b.

When programming in C, it is vital that consideration is taken to the type of variable when it is declared. There is considerable difference in the ranges of numbers able to be represented:

Bits	Max. unsigned	Max. signed	Min signed
N	$2^N - 1$	$2^{N-1} - 1$	-2^{N-1}
3	7	3	-4
8	255	127	-128
16	65535	32767	-32768
32	4294967295	2147483647	-2147483648

2 Other forms of data

Binary sequences are able to encode information other than numerical.

2.1 Hexadecimal

For easier human readability, consider hexadecimal notation:

- What is the maximum value of a 1-digit base-16 number?
- What is the largest number able to be encoded in a 4-bit sequence?
- How many bits do we represent with each single hexadecimal digit?

Decimal	Binary	Hex	Decimal	Binary	Hex
0	0000		8	1000	
1	0001		9	1001	
2	0010		10	1010	
3	0011		11	1011	
4	0100		12	1100	
5	0101		13	1101	
6	0110		14	1110	
7	0111		15	1111	

Just as a single binary digit within a sequence encodes the state of a single bit of memory, a single hexadecimal digit encodes exactly four bits. Just as we often use sequences of binary digits, we also use sequences of hexadecimal digits.

A four-bit sequence is half a byte: a *nibble*.

Hexadecimal encodes four binary bits, NOT a base-16 number!!.

Exercise: interpret the values in the table in 2's complement.

Decimal	Binary	Hex
19		
	`0b0101 1011`	
		`0xFF`

2.2 Floating point

Floating point representation is used for non-integer numbers. It uses the same theory as scientific number representation:

$$+4.232 \times 10^3$$

Binary scientific notation is used, where a 32-bit sequence is used, where bit 31 is S, bits 30 - 23 encode the exponent, and bits 22 - 0 encode the fractional part:



$$Value = (-1)^S \times 1.F \times 10^{(E-127)}_{(2)}$$

For example, the number $100.101_2 (= 4.625_{10})$:

$$= (-1)^0 \times 1.00101_2 \times 10^2_2$$

2.3 Characters

While numerical systems are important for being able to store data, it is also necessary to store text, and to do so at a very basic level. For example, the specific seat on an aeroplane, or different units of sublet houses are commonly stored in embedded systems. This is done by using a 7-bit sequence of binary, and assigning specific meanings to specific combinations. Broadly, there are two types of character encoded:

There are also 8-bit encodings, including an extended ASCII set, UTF-8, and UTF-16.

1. `0x00` - `0x1F`, and `0xFF` — Control codes

The most interesting of which is: `0x07`: 'Bell'.

2. `0x20` - `0x7E` — Printable characters

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0x	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1x	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2x	SP	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
3x	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4x	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5x	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6x	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7x	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Figure 2: ASCII table, where the y-axis is the first nibble, and the x-axis the second nibble.

We can use a sequence of characters (i.e. a sequence of sequences) to encode individual words.

3 Programming

What information is encoded in the following sequence?

0	1	0	1	0	1	0	1
0	0	1	1	1	0	0	0

If only there were some way of informing the computer exactly how to interpret each sequence. But this would be difficult, it would require *declaring* exactly the type of each variable. And you would have to do this before you referenced or wrote to the variable, so the computer knows exactly how to handle it.

Exercises:

1. What is the hexadecimal representation of: `0b1111 1110 1110 1101`?
2. A 32-bit computer has 512 words in its memory. How many bytes are there?
3. What is the value of: `(int8_t) ('z' + 42)`?
4. (extra) What is the 32-bit floating point representation of -23.25?

Summary

- Understand the need to store numerical values in computers.
  → The everyday world contains a large amount of continuous numerical information, and, given computers interact directly with the real world, computers must be able to handle numerical information.

- Understand how groups of logical variables are used to encode numerical information — binary.
  → As each digital variable can contain a single piece of digital information, the information we encode within each bit of a sequence is "does the sum total of the number include 2^n?", where n is the position of the digital variable/memory bit within the sequence:
  → $Value = \sum d_n \times 2^n$

- Understand how the representations of negative hexadecimal numbers correlate to digital information.
  → Negative numbers are typically stored as two's complement, where a `1` in the nth position shows a negative, and the value of a negative number is equal to:
  $Magnitidue = \sum_{n-1}(\overline{d_n} \times 2^n) + 1$
  → Hexadecimal is the base-16 number system, and is extremely useful because each digit is able to be directly, independently translated into its binary representation.
  → There is a method of storing non-integer numerical information in scientific notation.

- Understand the need to know which sequence is being used, and how this pertains to C programming.
  → We know that the interpretation of an 8-bit sequence can be vastly different depending on the interpretation: numbers can be signed integers, unsigned integers, or floats. Furthermore, 8-bit sequences can encode characters, or pointers.
  → In C, we declare the data type of a variable before we first use it, so the computer can be properly instructed how to handle it.

Next Lecture: Lecture 3 - Memory and buses

Questions?